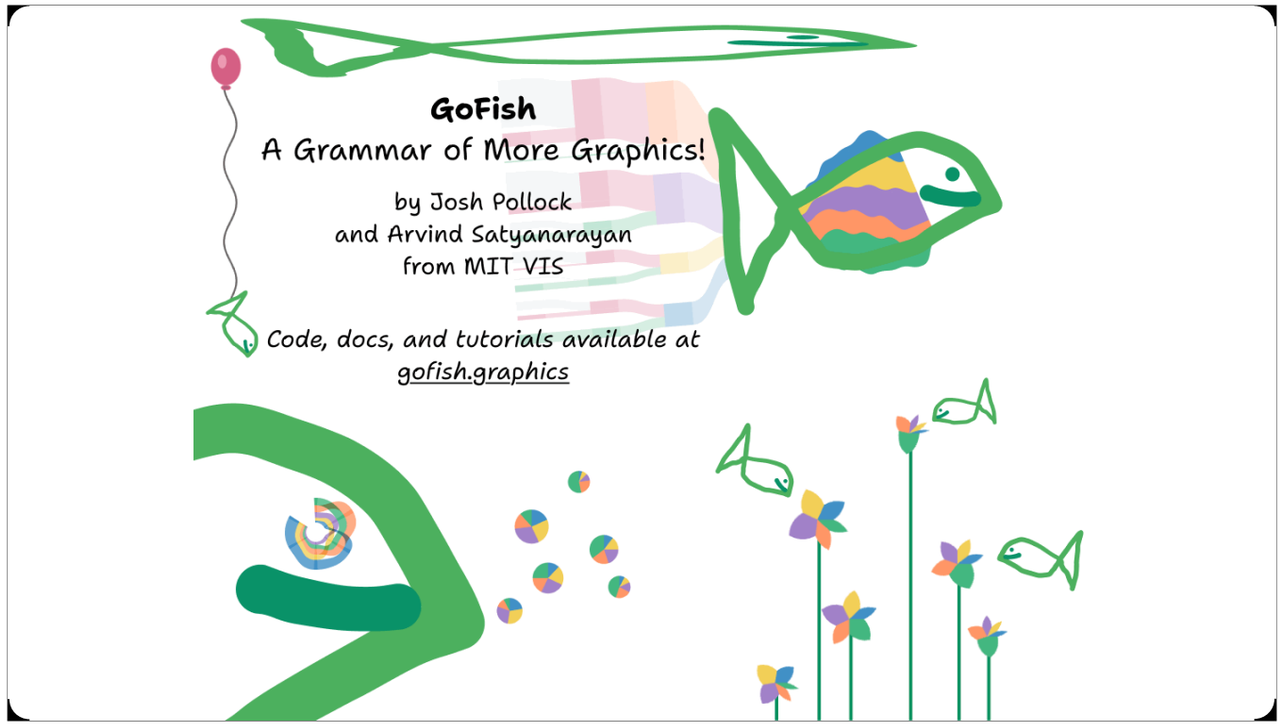
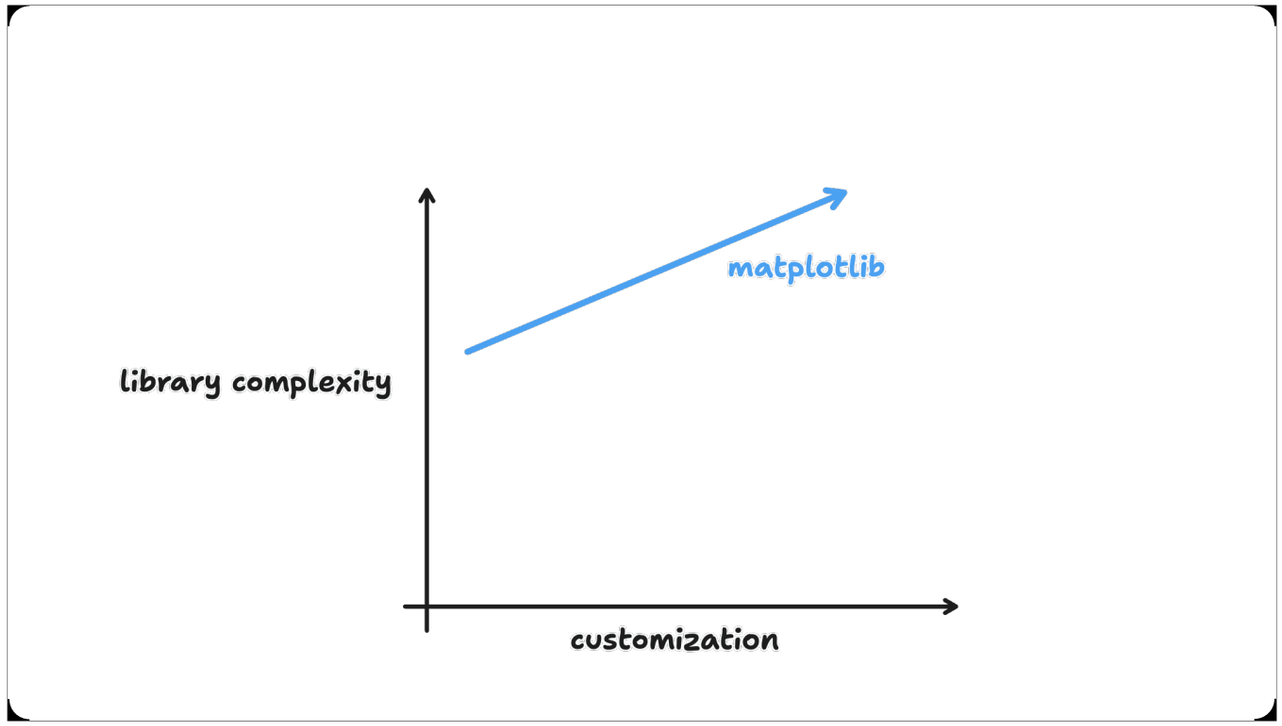
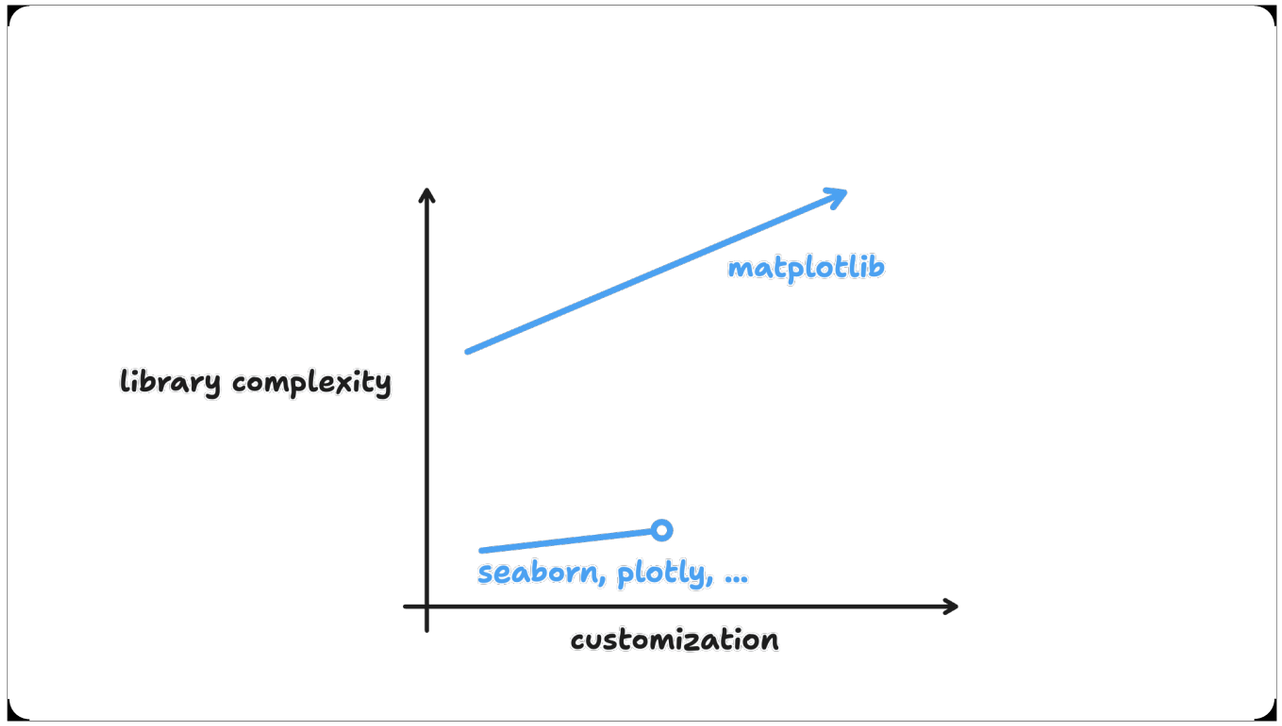
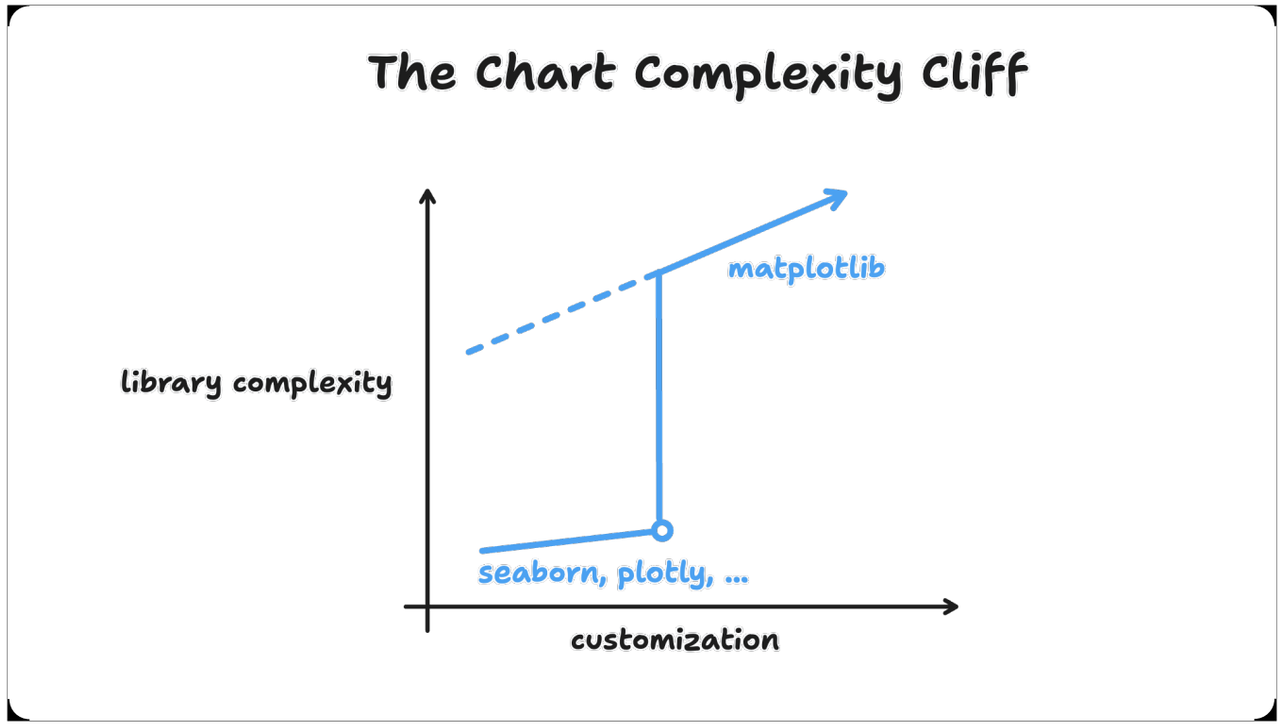

GoFish
A Grammar of More Graphics!
by Josh Pollock
and Arvind Satyanarayan
from MIT VIS
Code, docs, and tutorials available at
gofish.graphics



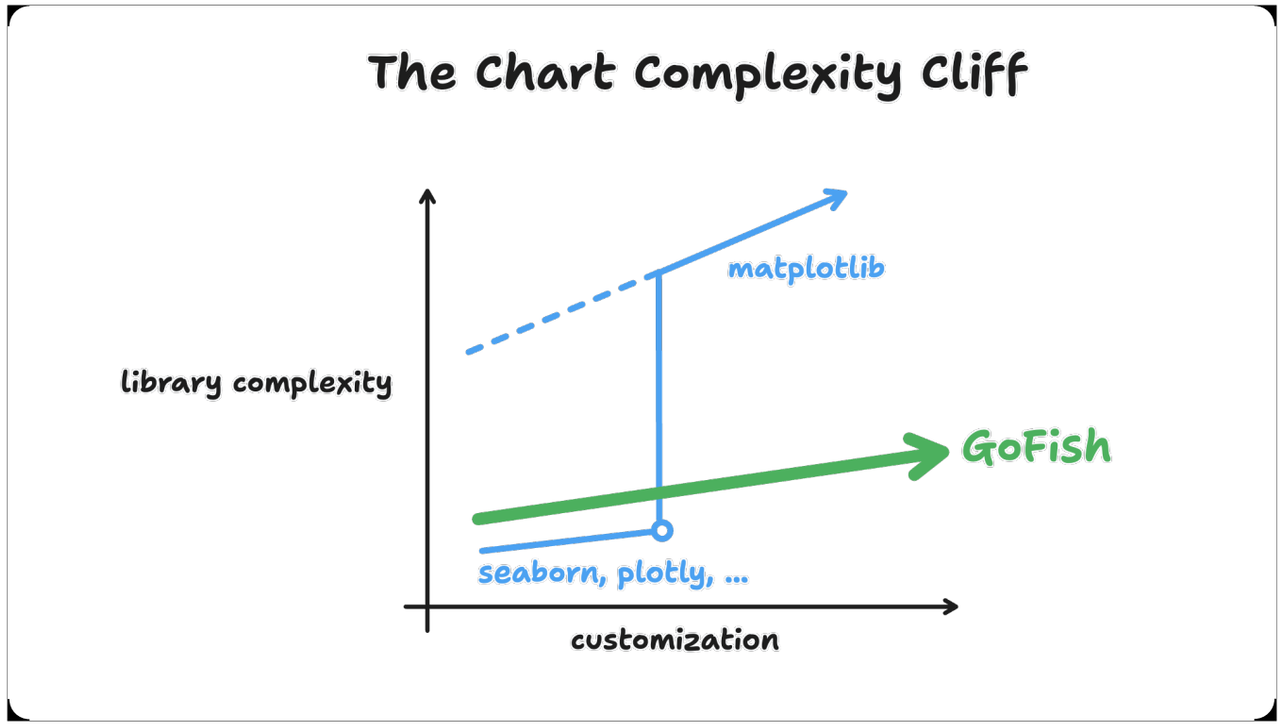

If we chart python charting libraries (hah) by how complex they are for the amount of customization we need. We might put matplotlib here. It's pretty complex, but let's you do basically anything.




Because it's so hard for even simple things, we turn to libraries like seaborn or plotly. But they severely limit what you can do.

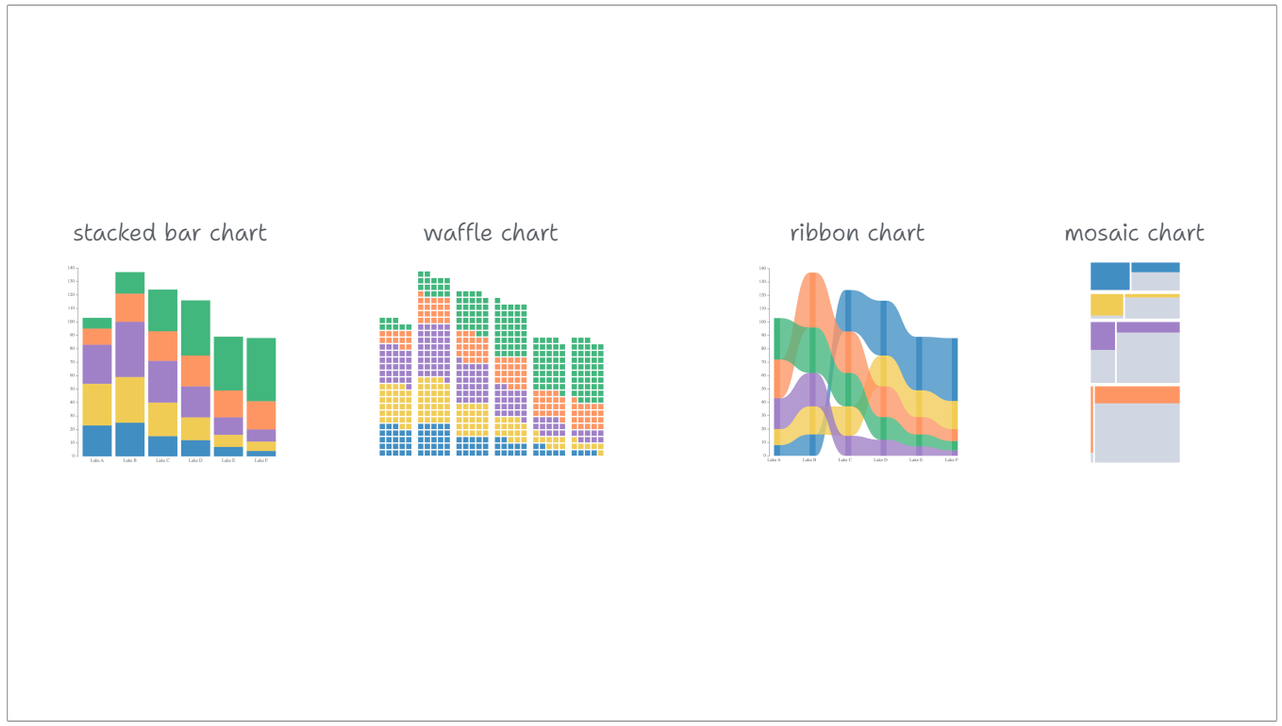



So we end up with this kind of chart complexity cliff. Once you hit a rough edge of one of these easy libraries, you have to bail out to matplotlib and eat all of that complexity.



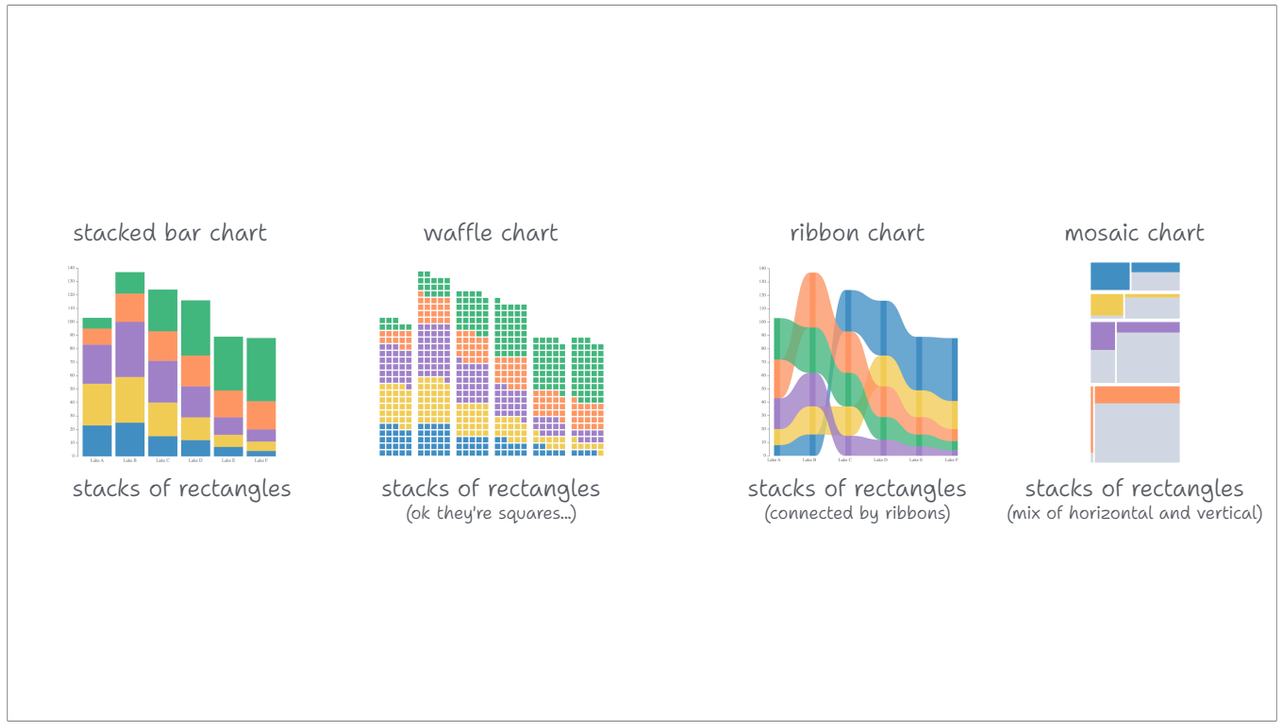

What if we had a charting library that was only a bit more complex than seaborn or plotly, but had the power of a library like matplotlib?

We're building a library like that. It's called GoFish and it'll be available soon for Python!

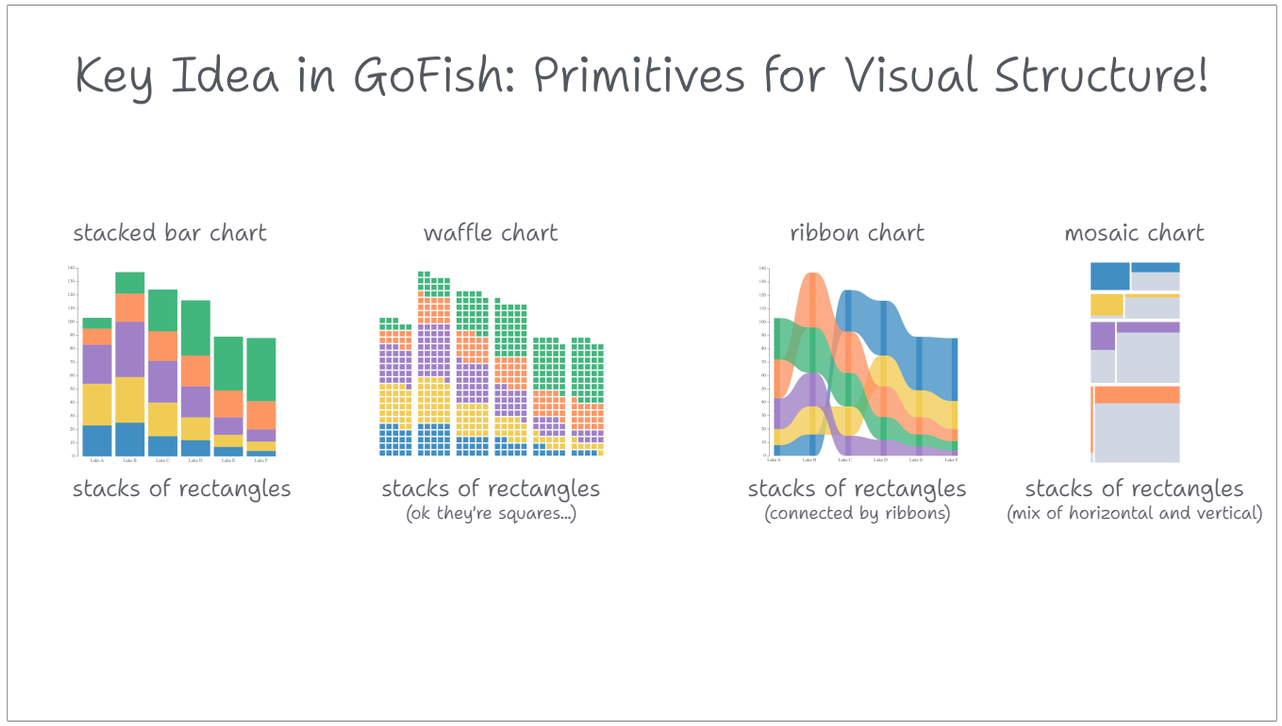



So how do we do that? Well take a look at these charts. We have a stacked bar chart, a waffle chart, a ribbon chart, and a mosaic chart. We're used to thinking of them as separate chart types, with separate API calls. But look closer!

These charts have a lot of visual structure! And a lot of it is shared!




A stacked bar chart is made of stacks of rectangles. So is a waffle chart, a ribbon chart, and a mosaic chart. They differ in how those stacks are arranged and whether they're connected with other shapes, and those differences are what make each chart unique and valuable, but they are composed in very similar ways.



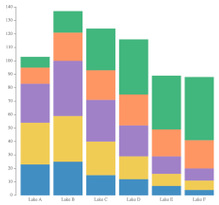

If we could describe charts directly in terms of that structure, we might be able to get rid of a lot of incidental complexity.

That's what we do in GoFish. We have primitives for this.

# Key Idea in GoFish: Primitives for Visual Structure!

## **graphical operators**

stacked bar chart

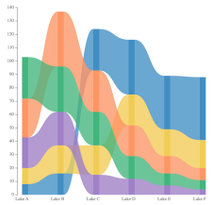


stacks of rectangles

waffle chart

stacks of rectangles
(ok they're squares...)

ribbon chart



stacks of rectangles
(connected by ribbons)

mosaic chart

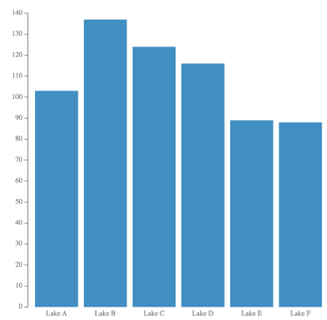
stacks of rectangles
(mix of horizontal and vertical)

And we call those primitives, *graphical operators.*

# GoFish: A Whirlwind Tour

So without further ado let's go on a whirlwind tour of GoFish!

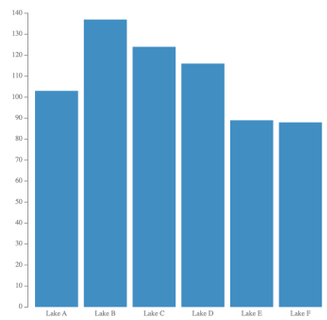
# Our First GoFish Spec




For our first spec, we're gonna make a bar chart! Now a basic spec in GoFish has three pieces that work in sequence.

# Our First GoFish Spec

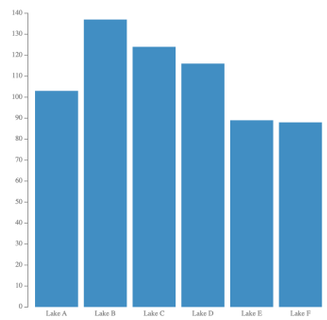
```
chart(seafood)
```




The first is the `chart` function.

# Our First GoFish Spec

```
data   chart(seafood)
```

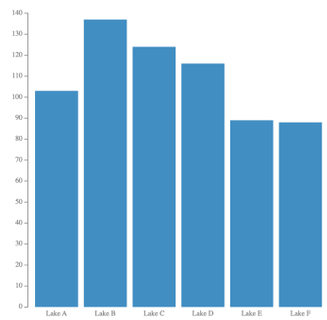



This is where we put the data we're going to use for our chart.

# Our First GoFish Spec

```
data   chart(seafood)
```

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
...
]
```

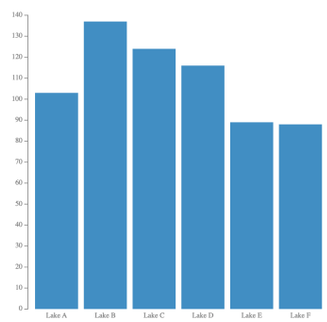



In this case we're using a dataset called seafood, which is an array of tuples with lakes, species of fish in those lakes, and how many of each species we caught.

# Our First GoFish Spec

```
data   chart(seafood)
         .flow(spread("lake", dir="x"))
```

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```

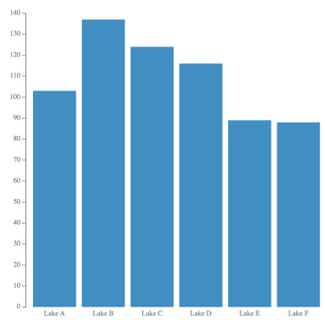



Next we add a graphical operator using the `flow` method.

# Our First GoFish Spec

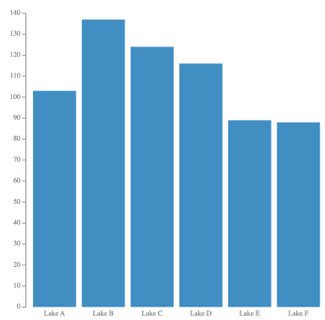
```
data      chart(seafood)
operators .flow(spread("lake", dir="x"))
```

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```

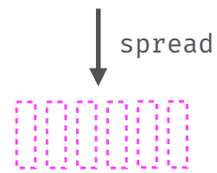


# Our First GoFish Spec

```
data      chart(seafood)
operators .flow(spread("lake", dir="x"))
```



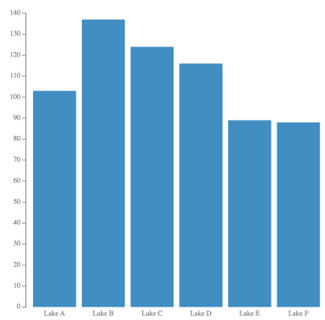

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```



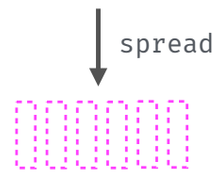

This operator, spread, will create a spread of regions, one for each of the six lakes in the dataset, in the x direction.

# Our First GoFish Spec

```
data      chart(seafood)
operators .flow(spread("lake", dir="x"))
          .mark(rect(h="count"))
```



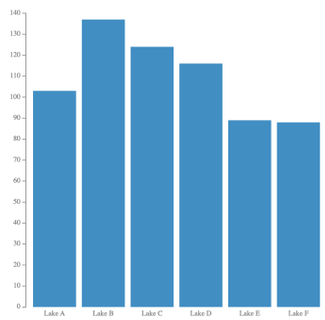

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```



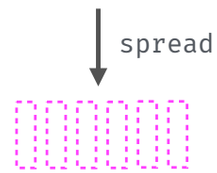

Finally, we'll add a mark, rect, to create a shape for each of those regions.

# Our First GoFish Spec

```
data      chart(seafood)
operators .flow(spread("lake", dir="x"))
mark      .mark(rect(h="count"))
```

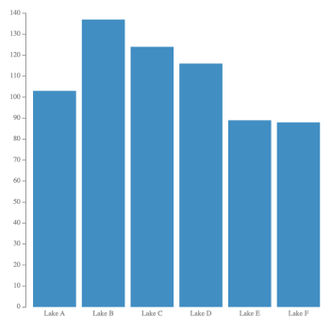



```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```

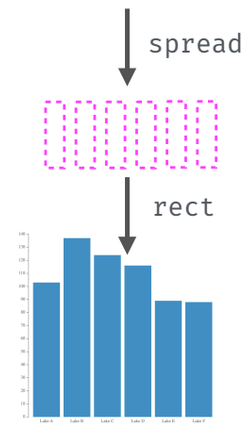


# Our First GoFish Spec

```
data      chart(seafood)
operators .flow(spread("lake", dir="x"))
mark      .mark(rect(h="count"))
```



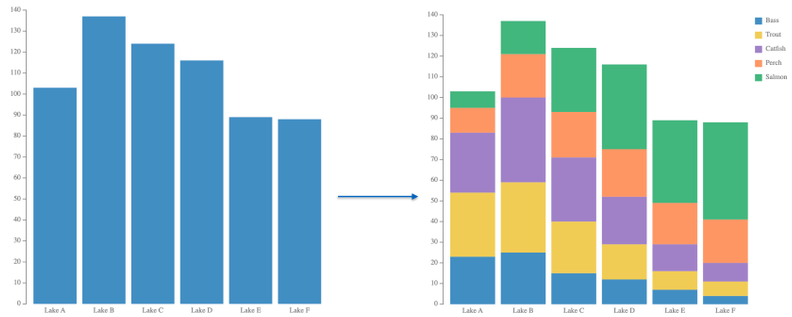

```
seafood = [
  {
    lake: "Lake A",
    species: "Bass",
    count: 23,
  },
 ...
]
```




The rect mark's encoding channel `h` tells GoFish to map the count of total fish caught in each lake to the height of the rectangle.

Ok that's a nice chart, but what if we want to break it down by species?

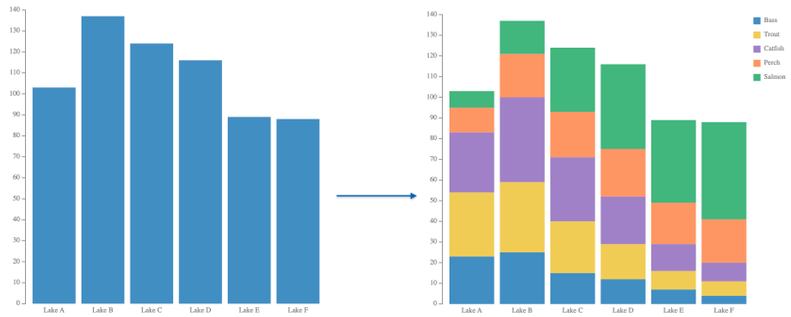
# Chaining Operators




We'll want to convert our bar chart into a stacked bar chart, and we can do this by chaining multiple graphical operators together.

# Chaining Operators

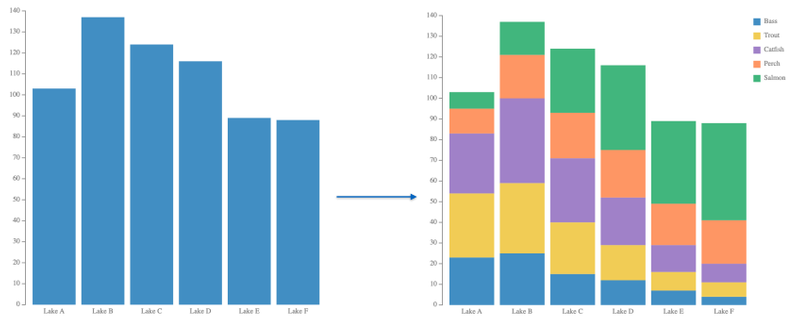
```
chart(seafood)
  .flow(spread("lake", dir="x"))
  .mark(rect(h="count"))
```




So starting with our bar chart spec

# Chaining Operators

```
chart(seafood)
  .flow(
    spread("lake", dir="x"),
    stack("species", dir="y")
  )
  .mark(rect(h="count", fill="species"))
```

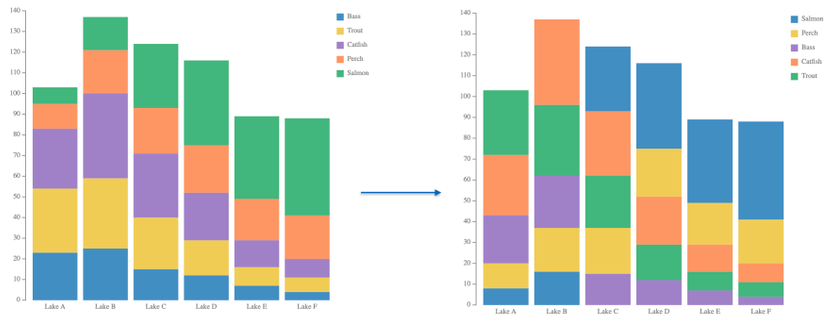



We'll add another graphical operator, stack, to create a vertical stack of rectangles inside each lake we created.
Stack is pretty similar to spread, except it doesn't add any spacing.

We'll also color each rectangle by its species.

Ok great! Hmm now that I see this chart, I'd actually like to see which species is the most abundant in each lake at a glance. It would be nice to sort all my bars!

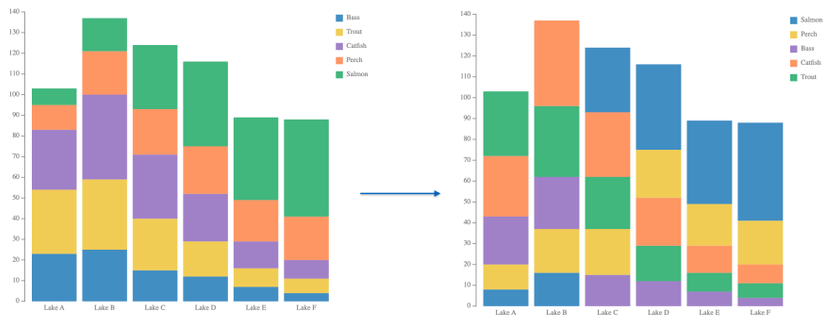
# Data Transforms




To do that, we'll use a data transform

# Data Transforms

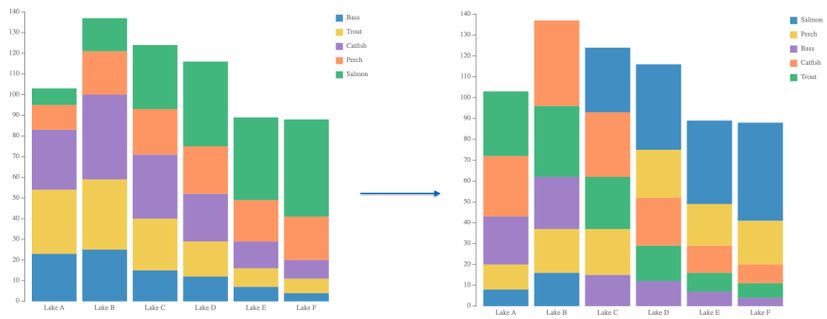
```
chart(seafood)
  .flow(
    spread("lake", dir="x"),
    stack("species", dir="y")
  )
  .mark(rect(h="count", fill="species"))
```




Because we've separated our chart's construction into four steps --- data, spread, stack, and rect --- we have a lot more control over when and how data transforms happen than we typically do in something like seaborn. As a result

# Data Transforms

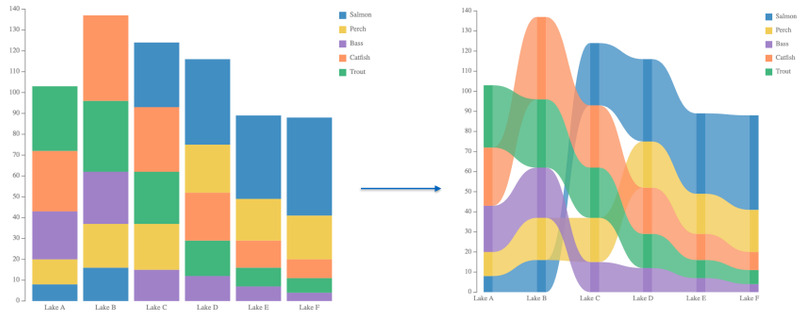
```
chart(seafood)
  .flow(
    spread("lake", dir="x"),
    derive(lambda d: d.sort_values("count")),
    stack("species", dir="y")
  )
  .mark(rect(h="count", fill="species"))
```




we can just insert a special "derive" operator into our flow, which will let us write any data transformation we want after the spread but before the stacking.

So now I can pretty easily tell that trout is the most abundant in Lake A, Catfish in Lake B, and Salmon in the rest. But it's harder to tell how these values are changing over time.

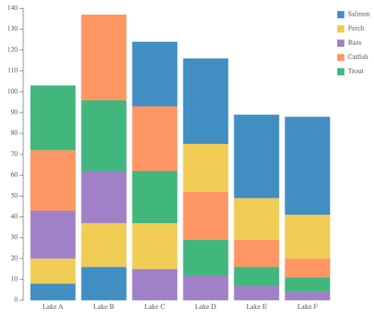
# Layering and Selection




I'd like to connect bars of the same species to make a ribbon chart. Now a ribbon chart is kinda funny, because it's a blend between a stacked bar chart and a stacked area chart. It's hard to describe that in a high-level library, but in GoFish we can make that connection precise. Let's see how.

# Layering and Selection

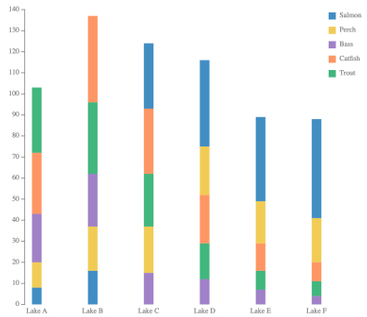
```
chart(seafood)
  .flow(
    spread("lake", dir="x"),
    derive(lambda d: d.sort_values("count")),
    stack("species", dir="y")
  )
  .mark(rect(h="count", fill="species"))
```




So starting with our sorted, stacked bar chart

# Layering and Selection

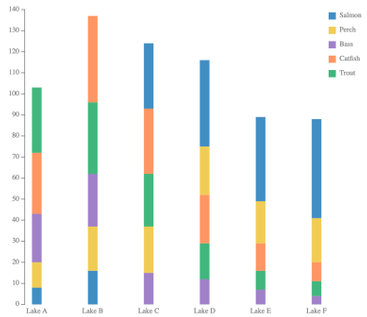
```
chart(seafood)
  .flow(
    spread("lake", dir="x", spacing=64),
    derive(lambda d: d.sort_values("count")),
    stack("species", dir="y")
  )
  .mark(rect(h="count", fill="species"))
```




We'll first adjust the spacing to make our bars smaller and provide some room for our ribbons

# Layering and Selection

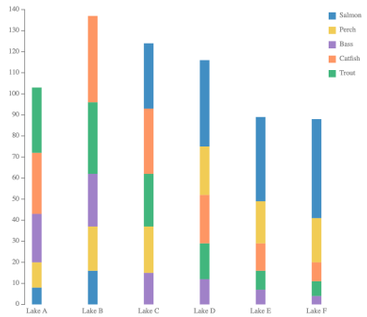
```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
])
```




Next we'll wrap our chart in a layer operator, since we're going to add the ribbons as a second layer

# Layering and Selection

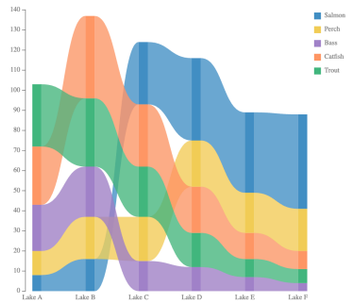
```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars")
])
```




And we'll name this layer "bars" so we can refer to it later.

# Layering and Selection

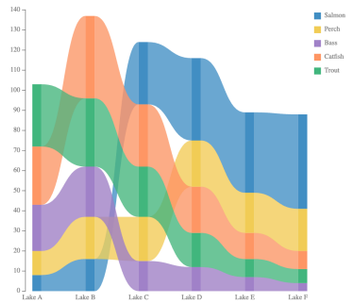
```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(???)
    .flow(???)
    .mark(???),
])
```




Now we'd like to add another chart call to complete this. Let's fill in each of the three steps.

# Layering and Selection

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(???)
    .mark(???),
])
```

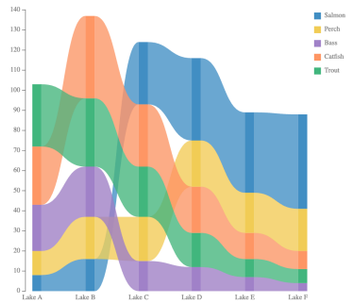



Our second layer is really *connecting* marks from the first layer, so we'll use this special `select` function to get those marks from the "bars" layer.

# Layering and Selection

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(???)
    .mark(???),
])
```

```
select("bars") =
[
  {
    lake: "Lake A",
    species: "Trout",
    count: 31,
    __ref: <GoFishMark>,
  },
...
]
```

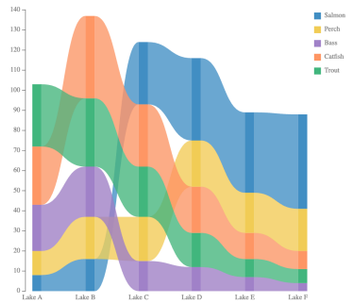



Select returns that layer's dataset enriched with references to the marks produced by the data

# Layering and Selection

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(???),
])
```

```
select("bars") =
[
  {
    lake: "Lake A",
    species: "Trout",
    count: 31,
    __ref: <GoFishMark>,
  },
...
]
```

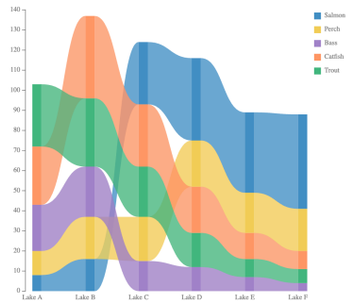



Next we'll group the dataset by species, since we want to make one ribbon per species

# Layering and Selection

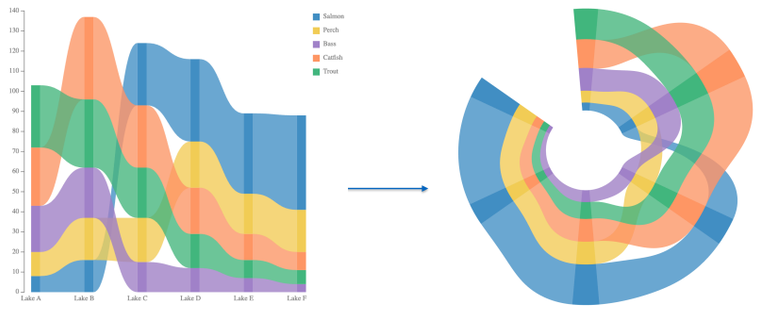
```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(area(opacity=0.8)),
])
```

```
select("bars") =
[
  {
    lake: "Lake A",
    species: "Trout",
    count: 31,
    __ref: <GoFishMark>,
  },
...
]
```




And finally we'll use an area mark that's slightly transparent to connect these bars. This mark is a bit special, in that it uses the __ref field as its inputs if they exist in the dataset. It sort of works like a hybrid between a mark and a graphical operator.

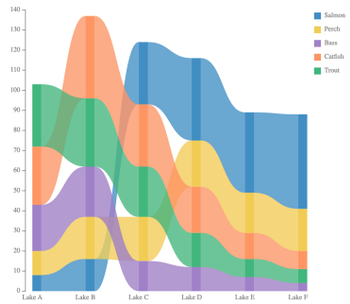
# Coordinate Transforms




That chart was nice... but let's be honest! Sometimes I just want to make something cool like this polar ribbon chart and we can use a coordinate transform for that.

# Layering and Selection

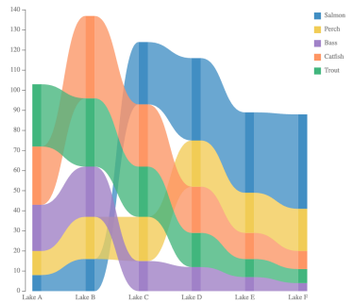
```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(area(opacity=0.8)),
])
```




The first thing I'll do is

# Coordinate Transforms: Polar Ribbon Chart

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=64),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(area(opacity=0.8)),
], coord=clock())
```

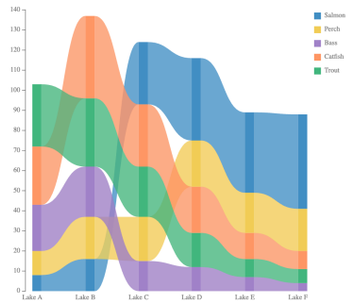



The first thing I'll do is add a coordinate transform to the layer.

The clock transform works like a clock, so it starts at 12'o'clock and goes clockwise.

# Coordinate Transforms: Polar Ribbon Chart

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=(2 * math.pi) / 6, mode="center", y=50),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(area(opacity=0.8)),
], coord=clock())
```




And then I'll change some of the properties on my spread so that it looks nice in polar space.

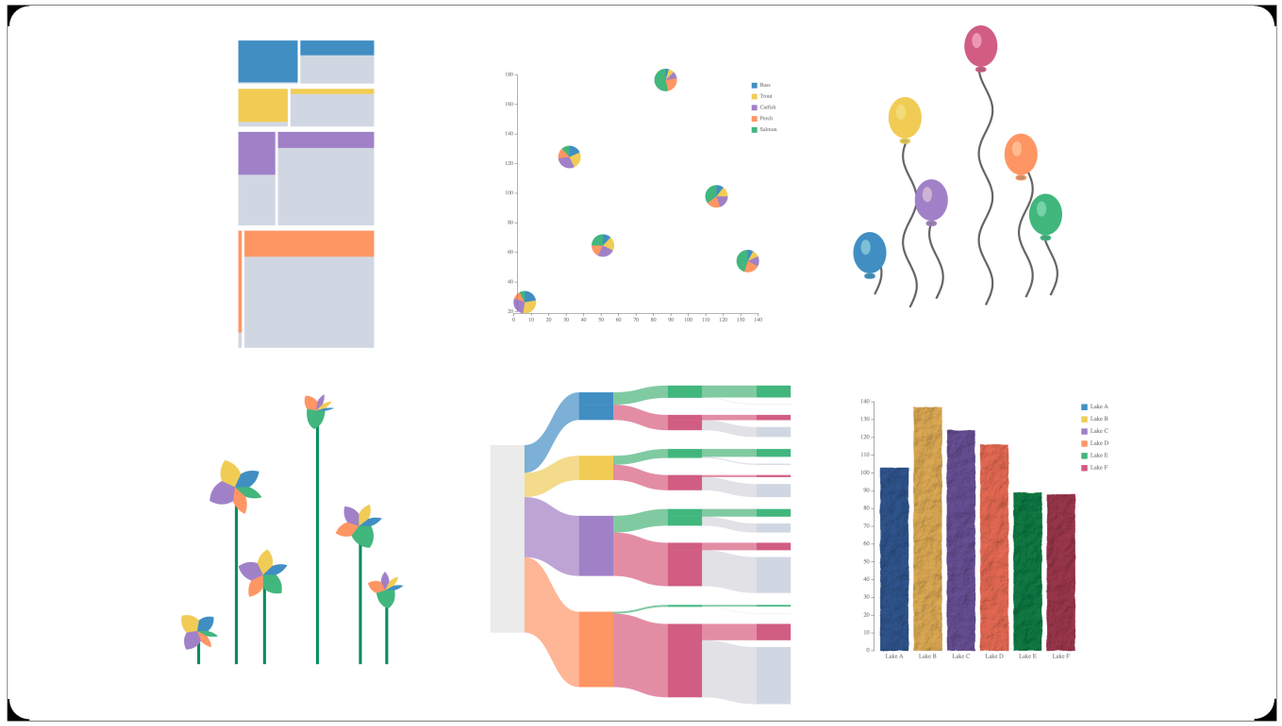
# Coordinate Transforms: Polar Ribbon Chart

```
layer([
  chart(seafood)
    .flow(
      spread("lake", dir="x", spacing=(2 * math.pi) / 6, mode="center", y=50),
      derive(lambda d: d.sort_values("count")),
      stack("species", dir="y")
    )
    .mark(rect(h="count", fill="species"))
    .name("bars"),
  chart(select("bars"))
    .flow(group("species"))
    .mark(area(opacity=0.8)),
], coord=clock())
```

And that's it! Now I have a polar ribbon chart!

So using graphical operators, we've made a system that is both concise and *really* expressive.

And it's not just polar ribbons




We can make mosaic charts. We can nest pie charts in scatter plots to make scatter pies. We can make balloon charts. We can make flower charts. We can make Sankey trees. We can apply filters to shapes.

# Future Work

- interaction/animation
- accessibility
- graphical operators for labeling, trees, networks, etc

**Sound interesting? Come talk to me!**

**I am looking for jobs to continue working on GoFish!**

jopo@mit.edu

Check it out! **gofish.graphics**

github.com/starfish-graphics/gofish-graphics

In the short to medium term, we're excited to explore how to add interaction and animation, how to make this much more larger set of charts accessible, how to add graphical operators for things like labels, trees, and networks, and adding a Python wrapper

If any of these projects sound interesting to you, please reach out!

Also I am looking for jobs to continue working on GoFish after I graduate in May! Please hire me